

信息工程学院本科生毕业设计说明书

题目： 基于clauswitz引擎的游戏mod设计与实现

摘要

优秀的历史类型游戏在全球的缺乏，使得Paradox Interactive（下称P社）在目前的市场上几乎没有相关的强力竞品，而这种状态会导致制作组逐渐摆烂。随着Europa Universalis 4利维坦DLC喜提steam差评榜第一，P社终于更换了新的制作组，开始更新全新的机制和接口，因此笔者制作了本mod《流浪拜占庭》，以满足网友的需求。

本mod采用Europa Universalis 4作为本体，运用P社提供的Clausewitz引擎接口进行设计，使用YAML进行本地化编写在游戏中实现汉语版本，使用Photoshop修改设计背景图片，使mod开发更合理、游戏运行更稳定，也降低了开发成本。通过综合考虑mod的剧情和流程，本mod分为转进和反攻两个部分。mod的主要玩法有：添加新的国家，为国家进行配套的设置如国家理念、任务树、相关事件等。同时对游戏的界面进行了重新设计，绘制了新的背景图片，更改其原本的布局使其更具人性化，同时也对一些原本的设计如商品、宗教、文化、时代等进行了符合本mod设定中的改进。

关键词：游戏； mod； Clausewitz引擎； YAML； Photoshop

目 录

1 绪论.....	1
1.1 课题研究背景.....	1
1.2 设计目的.....	1
1.3 mod 需求分析.....	1
1.3.1 mod 功能需求分析.....	1
1.3.2 可行性分析.....	2
2 系统的开发工具及技术简介.....	3
2.1 系统开发工具.....	3
2.1.1 visual studio code.....	3
2.1.2 ParaTranz Toolbox.....	3
2.2 系统开发的技术支持.....	3
2.2.1 Clausewitz 引擎.....	3
2.2.2 YAML.....	4
2.2.3 Photoshop.....	4
3 总体设计.....	5
3.1 mod 结构.....	5
3.1.1 流程图.....	5
3.1.2 模块结构图.....	5
3.2 游戏内容设计.....	6
3.3 游戏界面设计.....	6
3.3.1 窗口（windowType）.....	6
3.3.2 图标（iconType）.....	6
3.3.3 按钮贴图（guiButtonType）.....	6
3.3.5 滚动条（scrollbarType）.....	7
3.4 界面示例.....	7
3.4.1 政府界面.....	7

3.4.2 理念界面	7
3.4.3 时代界面	8
3.4.4 省份界面	8
3.4.5 阶层界面	9
3.5 相关代码	9
3.5.1 政府界面	9
3.5.2 理念界面	10
3.5.3 时代界面	11
3.5.4 省份界面	12
3.5.5 阶层界面	13
4 mod 功能实现	14
4.1 登记 mod	14
4.2 mod 基础设计	14
4.2.1 国家文件设计	15
4.2.2 国家历史设计	17
4.2.3 国旗	18
4.3 本地化	18
4.4 任务树设计	19
4.4.1 通用任务树	19
4.4.2 国家专用任务树	19
4.4.3 本地化	20
4.5 事件设计	20
4.5.1 事件类型	20
4.5.2 事件 id	20
4.5.3 标题和描述	21
4.5.4 事件图片	21
4.5.5 触发条件	22
4.6 决议	25

4.6.1 决议标题	26
4.6.2 潜在条件	26
4.6.4 决议效果	26
4.6.5 ai 意愿	27
4.6.6 本地化	27
4.7 灾难	27
4.7.1 灾难名称	28
4.7.2 潜在条件	28
4.7.3 开始条件	28
4.7.4 终止条件	28
4.7.5 灾难的月度增加	28
4.7.6 结束条件	29
4.7.7 灾难事件	30
4.7.8 月度事件	30
4.7.9 本地化	30
4.7.10 灾难图标	30
4.8 项目代码	31
4.8.1 任务树代码	31
4.8.2 事件代码	31
4.8.3 决议代码	32
4.8.4 灾难代码	33
5 mod 测试	35
5.1 功能测试	35
5.2 稳定性测试	39

1 绪论

1.1 课题研究背景

Paradox Interactive基于Clausewitz引擎制作了数量繁多的历史模拟类游戏，这些游戏以其优秀的兼容性和独特的游戏性而出名。因为该引擎旨在向希望修改原始游戏文件以创建mod的任何人开放，所以依托此类游戏所进行的mod制作相对简单。

在Paradox Interactive所制作的几款游戏中，笔者最终选择根据欧陆风云4（Europa Universalis 4 简称为EU4）进行mod开发。它所模拟的历史是以中世纪末期到法国大革命前夕之间以欧洲为主的历史，这段历史对应到中国便是明朝前期到清朝中期这段时间，相对不是很敏感。这款游戏的mod制作在国内也是如火如荼，前有对明代事件扩展的《天朝日不落》，后有专注于优化游戏体验的《女仆之国》。在群雄环聚的情况下，本mod打算由浅入深逐步推进mod内容，以求和上述mod竞争。

1.2 设计目的

本次mod制作是基于欧陆风云4这一款大战略游戏。游戏大部分设定都基于欧洲早期探索并殖民新世界的历史时期。

笔者在此mod里设计了一个与历史完全不同的世界线：假设历史上本应在1453年5月29日完全灭亡的东罗马帝国——拜占庭的难民逃难到爱尔兰岛上，重新复兴罗马。满足一些玩家的幻想：和汉朝一样伟大的罗马，能够顺利地传承下去，而不是淹没在历史的长河里。笔者为了满足网友对近几个游戏版本摆烂设计的不满，也尝试从这个过程中锻炼自己的能力。希望能由浅入深地展现一个全新的罗马，讲好一个成人童话。

1.3 mod需求分析

1.3.1 mod功能需求分析

本mod根据UP主资深小狐狸在2019年5月9日到2019年6月6日在哔哩哔哩视频网站所上传的视频《带着君堡去流浪》系列作为背景，因此将mod命名为《流浪拜占庭》

基于对玩家的调查研究和对上述视频的剧情致敬，mod需要满足以下的需求：如何让拜占庭人逃出君士坦丁堡到达爱尔兰，到达爱尔兰后对邻近的苏格兰和英格兰以及正处于英法百年战争的法兰西有什么影响？拜占庭难民如何统一爱尔兰，走向重建罗马之路。笔者为了完成mod向网友征求意见，特意将一些网友作为彩蛋加入其中，根据游戏情况添加各类修正，编写用于引导玩家的任务树，通过事件和游戏机制调整其他国家对待拜占庭的态度……总而言之，在处于mod的早期阶段时，大多是一些重复性的工作如任务树的搭建、事件逻辑的编写等。

1.3.2 可行性分析

可行性分析是对资源的合理使用，也是项目能够顺利进行的必要保证。下面从技术可行性和经济可行性两个角度来分析：

(1) 技术可行性

本mod是基于使用Clausewitz引擎的游戏实现，Clausewitz引擎具有相当开放的兼容性，允许任何人对游戏文件进行修改以制作“mod”。对图像的设计使用的是Photoshop和dds转换器，游戏原版并没有中文版本，所以可以进行中文的本地化来实现汉语的游戏界面。在码云和steam的创意工坊中都可以通过上传保存mod文件。因此在技术上是可行的。

(2) 经济可行性

本mod使用的软件除了游戏都是免费的，而游戏也可以通过学习版（只购买本体，DLC通过解锁补丁使用）或者破解版游玩，因此费用极少，所以在经济上是可行的。

2 系统的开发工具及技术简介

2.1 系统开发工具

《流浪拜占庭》mod使用Clausewitz引擎作为底层代码，运用visual studio code、ParaTranz Toolbox等开发工具进行开发，采用YAML编码技术进行可视性调整，以下作出详细介绍。

2.1.1 visual studio code

visual studio code（简称“VS Code”）是Microsoft在2015年4月30日Build开发者大会上正式宣布一个运行于 Mac OS X、Windows和 Linux 之上的，针对于编写现代Web和云应用的跨平台源代码编辑器，可在桌面上运行，并且可用于Windows，macOS和Linux。它具有对JavaScript，TypeScript和Node.js的内置支持，并具有丰富的其他语言（例如C++，C#，Java，Python，PHP，Go）和运行时（例如.NET和Unity）扩展的生态系统。

CWTools是一个用于VS Code的扩展，能够为mod的编写提供拼写验证、代码补全、语法高亮以及更多的功能。（在VS Code扩展中心搜索便可安装CWTools）

2.1.2 ParaTranz Toolbox

ParaTranz Toolbox是一款专门用于Clausewitz引擎mod制作的软件，可以将UTF-8编码转换为可被Clausewitz引擎识别的UTF-8-BOM编码。主要用于将代码进行对应的本地化，提升mod的观赏性和代码的可读性，由于是工具箱的形式，在Windows、macOS中均可使用，操作简单易于上手。

因为原版游戏使用的是单字节编码，所以要打上双字节补丁用来支持汉语的本地化，而双字节汉字在UTF-8-BOM编码中可读性极差，所以在本说明书中的本地化相关，大多是未转码前的状态。

2.2 系统开发的技术支持

2.2.1 Clausewitz引擎

Clausewitz引擎是Paradox Interactive（下称P社）自主研发的用于支撑游戏运算的引擎。Clausewitz引擎具有卓越的可读性、可见性、平台移植性和安全性，主要应用于Paradox Interactive的自研游戏。Clausewitz引擎编程语言的风格十分接近C语言、C++语言。

Clausewitz引擎是一个非开源的系统，一般所能了解的相关代码逻辑以及对其界面与内容所进行的修改，都是基于游戏目前所放出的代码逻辑，超出已有代码的范围便有可能导致全游戏的崩溃。当游戏进行版本迭代时，可能会放出部分新代码，用于支援mod的开发。

在mod应用开发中，可提供编辑的项目有很多，但主要集中在以下部分：用

于动态地影响游戏、用于执行命令/事件/决议等机制、只有当某些条件达到时才会触发的“event”、用以引导游戏过程并展现不同特色的“mission”。在这些脚本的编辑中,Paradox凭借其详实可观的系统触发与指令效果使制作mod的难度大大减少。

2.2.2 YAML

YAML是一个可读性高,用来表达数据序列化的格式。YAML参考了其他多种语言,包括:C语言、Python、Perl,并从XML、电子邮件的数据格式(RFC 2822)中获得灵感。Clark Evans在2001年首次发表了这种语言,另外Ingy döt Net与Oren Ben-Kiki也是这语言的共同设计者。当前已经有数种编程语言或脚本语言支持(或者说解析)这种语言。

YAML的语法和其他高级语言类似,并且可以简单表达清单、散列表,标量等数据形态。它使用空白符号缩进和大量依赖外观的特色,特别适合用来表达或编辑数据结构、各种配置文件、倾印调试内容、文件大纲(例如:许多电子邮件标题格式和YAML非常接近)。它不仅适合用来表达层次结构式(hierarchical model)的数据结构,也有精致的语法可以表示关系性(relational model)的数据。由于YAML使用空白字符和分行来分隔数据,使得它特别适合用grep/Python/Perl/Ruby操作。其让人最容易上手的特色是巧妙避开各种封闭符号,如:引号、各种括号等,这些符号在嵌套结构时会变得复杂而难以辨认,适用于脚本语言,配置文件和序列化等场景。在本次mod制作中使用YAML语法配合ParaTranz Toolbox进行本地化的完善。

2.2.3 Photoshop

Adobe Photoshop,简称PS,是由Adobe Systems开发和发行的图像处理软件,该软件支持Windows操作系统、Android与Mac OS。Photoshop主要处理以像素所构成的数字图像。使用其众多的编修与绘图工具,可以有效地进行图片编辑工作。ps有很多功能,在图像、图形、文字、视频、出版等各方面都有涉及。在本次mod制作中Photoshop主要用于处理背景图片和新图标。

3 总体设计

3.1 mod结构

3.1.1 流程图

本mod主要基于游戏Europa Universalis IV所构建,将mod文件放置在我的文档\Paradox Interactive\Europa Universalis IV\mod中,再通过P社自带的游戏启动器来选择是否使用mod,如果在启动器中勾选:使用mod,就会使用本mod进行游戏。如果不勾选,则以游戏原貌进行游戏,流程图如图3-1所示:

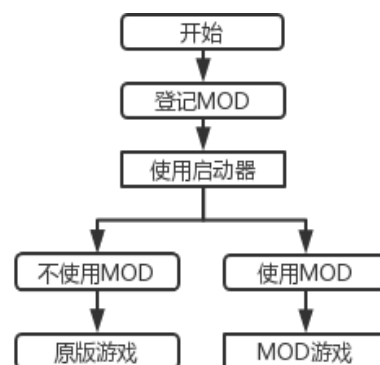


图3-1 流程图

3.1.2 模块结构图

此mod主要分为游戏内容和界面设计,游戏内容主要是对事件、决议、任务树、国旗国家、文化等脚本文件进行编译,游戏内容结构图如图3-2所示;界面设计主要是对游戏内界面进行改进,如政府界面、时代界面、理念界面等,界面设计结构图如图3-3所示。

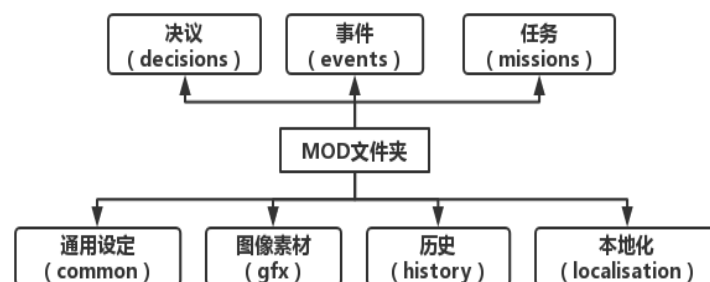


图3-2 游戏内容结构图

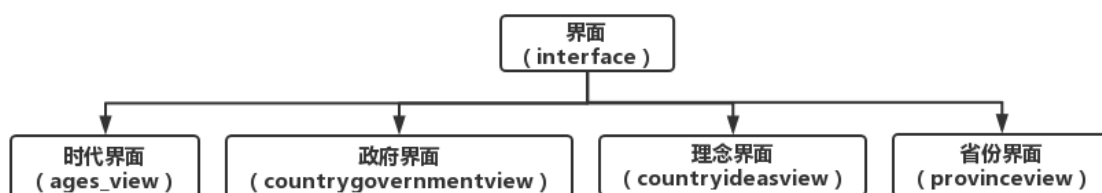


图3-3 界面设计结构图

3.2 游戏内容设计

在mod中通用设定（common）一般作为基底，为其余设计提供支持；图像素材（gfx）是配合界面设计，其中包含游戏内使用的艺术资源，一般沿用游戏本体资源即可，也可以在其基础上进行重新设计；历史（history）主要定义了游戏中每个国家情况、省份情况、国家间的外交情况；本地化（localisation）包含.yml文件，这些文件指定每一行代码在游戏中显示哪些文本，由于原版游戏是单字节英文编码，所以翻译成双字节中文相对困难。

决议（decisions）中的.txt文件定义了一个国家可以做出的决议，包括其前提条件，影响以及决定ai是否选择它的因素；事件（events）中的文件定义了玩家或ai国家可能发生的事件：触发条件、触发频率以及事件的选项和效果；任务（mission）定义玩家和ai可以尝试的任务，包括成功和失败的条件以及奖励。

3.3 游戏界面设计

界面文件主要由容器（container）和元素（elements）两部分组成。界面样式根据原版游戏的界面和mod自身需求进行重新设计，同时借鉴了一些其他mod里公认经典的界面设计。本次界面设计主要运用了五种类型，分别是窗口（windowType）、图标（iconType）、按钮贴图（guiButtonType）、可滚动列表框（listboxType）、滚动条（scrollbarType），以下做详细介绍。

3.3.1 窗口（windowType）

窗口是最常见的容器，可以用于容纳任何种类的容器。为了契合原本的美术风格，并没有过多修改，主要使用了下列属性：用于定义容器的屏幕放置位置的position，用于定义容器的边框大小的size，用于设置position属性坐标原点的orientation

3.3.2 图标（iconType）

图标中的元素用于向界面增加图标贴图，它的用法与guiButtonType基本相同，主要使用下列属性：用于定义图标的屏幕放置位置的position、用于定义position属性坐标原点的orientation、用于定义图标贴图的spriteType、用于定义使用多帧贴图时的frame。

3.3.3 按钮贴图（guiButtonType）

按钮贴图元素用于向界面增加按钮贴图。按钮可以被定义为图像和文本，因此它的生效方式类似于iconType和instantTextBoxType。主要使用下列属性：用于定义图标的屏幕放置位置的position、用于定义position属性坐标原点的orientation、用于定义图标贴图的spriteType、用于定义使用多帧贴图时的frame。

3.3.4 可滚动列表框（listboxType）

可滚动列表框用于向界面增加列表贴图，这指的是包括诸多具体条目的可滚动列表框。一般来说，这些elements与其他container中的internal代码相关联，该container主要用来定义列表框中使用的具体文本。主要使用下列属性：用于定义列表框的屏幕放置位置的position、用于定义position属性坐标原点的orientation、用于定义列表框的尺寸大小的size、用于定义列表框的滚动条的scrollbartype、用于定义滚动条是水平还是垂直的horizontal。

3.3.5 滚动条（scrollbarType）

滚动条一般用于和可滚动列表框（listboxType）配合使用，它们的用法相似，主要使用了以下的属性：用于定义滚动条的屏幕放置位置的position、用于定义滚动条的尺寸大小的size、用于定义滚动条相对于其他elements的优先级的priority、用于定义滚动条是水平还是垂直的horizontal。

3.4 界面示例

3.4.1 政府界面

将国家能力和国家修正交换了位置，这是因为在mod的设定里，一个国家可能会拥有多种政府能力，如果不进行修改的话会导致发生重叠，无法有效交互。因此将容纳国家能力的窗口和其中的按钮贴图单独提取出来，再把强化政府的按钮调整到单独的窗口，国家修正的窗口移到左下，在右侧将政府能力完全展现出来。原版如图3-4(1)本体界面所示，mod版本如图3-4(2)mod界面所示。



图3-4(1) 本体界面



图3-4(2) mod界面

3.4.2 理念界面

将底部漏出的理念组收纳进窗口中，并为理念组列表框优化了滚动条，使其更加美观，原版如图3-5(1)本体界面所示，mod版本如图3-5(2)mod界面所示。



图3-5(1) 本体界面



图3-5(2) mod界面

3.4.3 时代界面

由于原版游戏设计之处并没有多余的位置容纳多出的时代能力，所以笔者在mod里改进了背景图像设计，将时代能力移至左侧，其余元素右移，以方便进行拓展。本体界面如图3-6(1)所示，mod版本如图3-6(2)所示。



图3-6 (1) 本体界面



图3-6 (2) mod界面

3.4.4 省份界面

在开发中笔者对原版游戏中的商品进行了增加，这个设计除了新增图标外，还要修改图标的分配方法，所以同时更改贴图 and 文件，如果不进行更改，对产物图标的分配将出现差错。原版如图3-7(1)本体界面所示，修改版本如图3-7(2)mod

界面所示。



图 3-7(1) 本体界面



图 3-7(2) mod 界面

3.4.5 阶层界面

在原版的游戏中，每个阶层最多拥有4个特权槽位（在新版本里似乎扩展到了5个），在本mod中为每个阶层多添加了一些特权槽位。并改变了原本的界面布局，原版如图3-8(1)本体界面所示，修改版本如图3-8(2)mod界面所示。



图 3-8(1) 本体界面



图 3-8(2) mod 版本

3.5 相关代码

3.5.1 政府界面

为了显示多种政府能力，要在GUI文件中对布局进行修改。将显示特殊政府能力集成到一个列表栏位中（ListboxType），每个特殊政府机制的窗口都以此作为坐标起点显示。

将显示特殊政府机制的窗口（windowType），从界面的左下移动到右上。而存在列表中的特殊政府能力也要调整相应数值的X轴。因为改变了元素的位置，所以也要把在GFX文件夹中的背景图片更换为图3-9政府界面背景。



图 3-9 政府界面背景

关键代码如下：

```
listboxType = {
    name = "specific_governments_list"
    position = { x = 190 y = 220 }    #x = 10 y = 220
    size = { x = 360 y = 100 }
    #position = { x = 600 y = 0 }
    #size = { x = 400 y = 1000 }
    scrollbarType = "standardlistbox_slider"
}
#更改特殊政府能力列表框的位置
windowType = {
    name = "states_general"
    backGround = ""
    position = { x=180 y=173 }    #x=0 y=273
    size = { x=280 y=50 }
    #跟随列表框的位置变动
}
windowType = {
    name = "russian_mechanic"
    backGround = ""
    position={ x = 0 y = 0}
    size = { x = 300 y = 100 }
    #以specific_governments_list的位置作为起点所以无需更改position
}
```

3.5.2 理念界面

因为原版游戏并没有设计超出行数的理念组，导致滚动条的设置比较随意，所以为了容纳两根滚动条，要调整理念组的位置与体积，从而扩大每个理念组的间距，确保滚动条不会因此出现贴图错误。

关键代码如下：

```
listboxType = {
    name = "ideagroups_listbox_left"
    backGround = ""
    position = { x=13 y=243 }    # x=25 y=243 原来的位置
    size = { x=248 y=340 }    # x=270 y=420 原来的体积
    #更改列表框的大小使其和背景图片相同，列表框滚动条就不会错位
}
listboxType = {
    name = "ideagroups_listbox_right"
    backGround = ""
    position = { x=275 y=243 }    #x=276 y=243 原来的位置
    size = { x=248 y=340 }    #x=270 y=420 原来的体积
    #更改列表框的大小使其和背景图片相同，列表框滚动条就不会错位
}
```


3.5.3 时代界面

因为原版游戏中的时代能力数量是限定11项的，为了增加时代能力，要更改其布局。将所有元素向右移动一定的距离，随后将原本的时代能力列表从水平列表改为垂直列表，对应的改变其大小。为新添加的对象预留空间。由于更改了布局，也要在GFX文件夹中替换背景图片。如图3-10时代界面背景所示。



图 3-10 时代界面背景

关键代码如下：

```
windowType = {
    name = "ages_view"
    backGround=""
    position = { x = -290 y = -280 }
    size = { x = 512 y = 512 }

    windowType = {
        name = "age_of_discovery"
        position = { x = 148 y = -29 } # x= 35 y = -29向右移动123像素
        upsound = age_of_discovery
        #将时代主页面右移
    }
    listBoxType = {
        name ="abilities"
        position = { x = 5 y = 17 } # x = 4 y = 244
        backGround=""
        size = { x = 70 y = 660 } #x=600y=600方向改变了所以减少宽度
        horizontal = 0 # 1 从水平分布改为垂直分布
```

```
        #将时代能力列表框从原本的主页面中的位置分离，在左侧垂直分布
    }
}
```

3.5.4 省份界面

因为主要是对省份中的一个图标元素进行修改，所以并不用在GUI文件中更改结构，转而在GFX文件中修改。

在GUI文件中：第一个图标（iconType）代表了潜在的商品，是随着游戏进行可能会改变的商品。第二个图标（iconType）是省份当前的商品。它们会和resources_small.dds和resources.dds图片中的商品一一对应。

关键代码如下：

```
iconType = {
    name = "latent_resource_icon"
    spriteType = "GFX_resource_icon"
} #说明图标所对应的图片
guiButtonType = {
    name = "resource_icon"
    quadTextureSprite = "GFX_resource_icon"
} #说明图标所对应的图片
```

GFX文件中：将对应商品图片的栏位将原本的32件商品的分割方式，改变为33件。

```
spriteType = {
    name = "GFX_resource_icon"
    texturefile = "gfx//interface//resources.tga"
    noOfFrames = 33 # 32 原先只有32件商品
    loadType = "INGAME"
}
spriteType = {
    name = "GFX_resource_icon_small"
    texturefile = "gfx//interface//resources_small.dds"
    noOfFrames = 33 # 32
    loadType = "INGAME"
}
spriteType = {
    name = "GFX_resource_icon_transparent"
    texturefile = "gfx//interface//resources.tga"
    noOfFrames = 33 # 32
    loadType = "INGAME"
    alwaystransparent = yes
}
```

3.5.5 阶层界面

将阶层界面整体左移，阶层列表从原本占半个界面的宽度现在拓展到整个界面的宽度。提供特权的滚动条在列表界面变长后，原本被隐藏的按钮控件就显示出来了。由于改变了布局，要用新的阶层背景图片在GFX文件夹里进行更换。如图3-11阶层界面背景所示。



图 3-11 阶层界面背景

关键代码如下：

```
windowType = {
    name = "estate_item"
    position = { x= -30 y= 0 } # x=0 y=0
    size = { x=500 y=96 }
    moveable = 0
    Orientation = "UPPER_LEFT"
    listBoxType = {
        name = "privileges_list"
        backGround = ""
        position = { x = 34 y = 47 } # x = 5 y = 87
        size = { x = 500 y = 150 } # x = 240 y = 150
        Orientation = "UPPER_LEFT"
        horizontal = 1
        spacing = 0
        scrollbarType = "standardlistbox_invisible"
        borderSize = { x = 0 y = 0 }
    }
    #将原本的阶层背景拉长。一个阶层占据一行
    scrollbarType = {
        name = "privileges_scrollbar"
        size = {x =70 y =12 } #x =230 y =12
        position = {x=-50 y =70} #x= 10 y =145
        priority = 100
        borderSize = {x =16 y = 16}
        horizontal = 1
        #将阶级特权也随之拉长
    }
}
```

4 mod功能实现

4.1 登记mod

P社的游戏启动器是通过检查mod登记文件中的路径读取mod文件。使用相对路径较为便捷，因为本mod英文名为zishenfox，所以在.mod登记文件中写入以下代码：path="mod/zishenfox"。启动器就会在我的文档或游戏根目录下的mod文件夹中寻找对应的mod。所以一般推荐将mod放置在：我的文档\Paradox Interactive\Europa Universalis IV\mod的路径下。mod文件位置如图4-1所示，

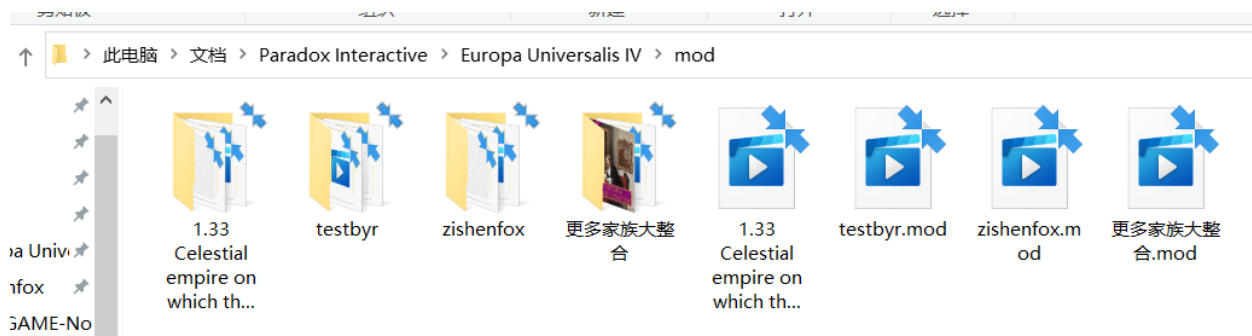


图4-1 mod文件位置

4.2 mod基础设计

制作一个中型mod，首先要创建一个新的国家tag，这个tag在游戏编辑中将代表所对应的国家。所有tag都应当是独一无二的，不能重复。在country_tags文件夹中的tag文件中添加需要的tag（如图4-2国家tag设计所示）。然后在countries文件夹下创建对应路径的国家文件。国家文件创建如图4-3所示。

```
zishenfox.mod  ZF_countries.txt
1  #FOX tags:#
2  BYR = countries/byreland.txt #拜尔兰
3  #SDF = countries/shieldFox.txt
4  #SHF = countries/shameFox.txt
5  #SFF = countries/sisterFox.txt
6  #FHF = countries/Foxhistory.txt
7  #FEF = countries/Foxemperor.txt
8  #FNF = countries/FoxNeutral.txt
9  NGV = countries/NorthGermanViceroyalty.txt #北日耳曼总督区
10 EGV = countries/EgyptViceroyalty.txt #埃及总督区
11 GLV = countries/GaulViceroyalty.txt #高卢总督区
12 AZA = countries/Aztecnia.txt #阿兹特克尼亚
13 MYI = countries/Mayania.txt #玛雅利亚
14 ICA = countries/Incania.txt #印加利亚
```

图4-2 国家tag设计

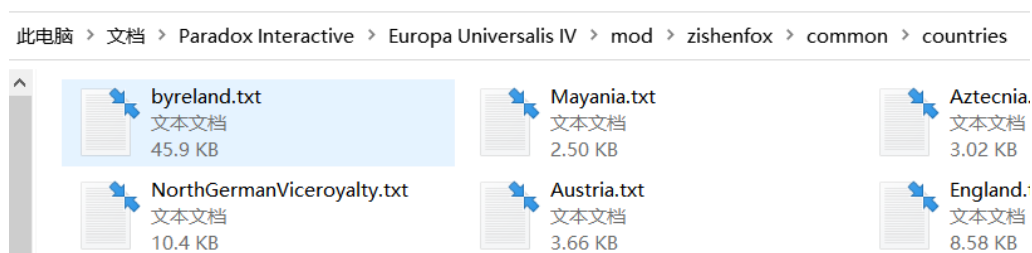


图 4-3 国家文件创建

4.2.1 国家文件设计

(1) 文化图像

文化图像指的是军队和顾问的图像，游戏本身有自带的素材库直接使用即可，代码如下：

```
graphical_culture = easterngfx
```

(2) 颜色

此处指的是在政治地图中显示的国家颜色，用RGB模式来表示，代码如下：

```
color = { 148 44 101 }
```

(3) 历史理念组

当ai操作这个国家时将会按照顺序选择这些理念组。当游戏开启时国家的行政科技允许国家解锁理念时，这个理念组合将会决定所开启的理念组。代码如下：

```
historical_idea_groups = {  
    exploration_ideas  
    defensive_ideas  
    religious_ideas  
    administrative_ideas  
    .....  
}
```

(4) 历史兵种

决定ai会选择陆军单位的具体种类，代码如下：

```
historical_units = {  
    eastern_medieval_infantry  
    eastern_knights  
    slavic_stradioti  
    eastern_militia  
    .....  
}
```

(5) 君主的名字

国家的君主的名字会在此列表中随机选择。名字后面的“#”代表的是该名字的代数，例如如果文件里是“Steve #2”，那么当国家政体类型允许的时候，下一个名字为Steve的君主会被命名为Steve III。“=[数字]”决定君主和继承人选择这个名字的概率。负数代表该名字为女性名字。

由于本地化的需求，所有的中文字体必须进行转码才能显示，为了可读性，本次展示的是转码之前的代码。代码如下：

```
monarch_names = {  
    "亨利 #3" = 100 # Henry #3  
    "爱德华 #3" = 60 # Edward #3  
    "查尔斯 #0" = 40 # Charles #0
```

```

"詹姆斯 #0" = 40 # James #0
"威廉 #2" = 20 # William #2
"理查德 #2" = 20 # Richard #2
"玛丽 #0" = -20 # Mary #0
"菲利普 #0" = 20 # Philip #0
"亨利 #3 弗雷德里克" = 10 # Henry #3 Frederick
"安妮 #0" = -10 # Anne #0
"亚瑟 #0" = 10 # Arthur #0
"查尔斯 #0 詹姆斯" = 10 # Charles #0 James
"埃德加 #0" = 10 # Edgar #0
"托马斯 #0" = 10 # Thomas #0
"埃德蒙 #0" = 5 # Edmund #0
"乔治 #0" = 5 # George #0
"约翰 #1" = 1 # John #1
"斯蒂芬 #1" = 1 # Stephen #1
"伊丽莎白 #0" = -10 # Elizabeth #0
"简 #0" = -10 # Jane #0
"玛格丽特 #0" = -1 # Margaret #0
}

```

(6) 将领的名字（舰船的名字）

这决定了将领（包括征服者、探险家等）与舰船的名字，对包含多个单词的名称使用引号。本地化同样要经过转码才可应用，在mod设计阶段提供部分援助的一些网友的要求下，在这个mod中新建的国家大多是使用网友的名字，代码如下：

```

leader_names (ship_names) = {
    " 残叶断梦 #0" = 10
    " 泳霖大大 #0" = 10
    " RTTPRL #0" = 10
    " 浅雨墨夜 #0" = 10
    " 109 #0" = 10
    " OLDCADER #0" = 10
    " Soledad #0" = 10
    " 风花镜月 #0" = 10
    " 瓜丁Guading #0" = 10
    " 鹿 #0" = 10
    " 尼采的救赎 #0" = 10
    " 牛蛙牛蛙 #0" = 10
    " 青空sama #0" = 10
    " 清风白月 #0" = 10
}

```

4.2.2 国家历史设计

在/history/countries中创建一个文档。也可以选择复制、重命名、修改一个类似的文档。文档名应该为: [国家tag - 国家名], 例如BYR - Byreland"。这个文档包含这个国家的历史以及默认的各类信息。

(1) 政体

这里可以选择为任一种政体以及政府等级。 1.26版本重写了政体机制, 同种政体类型不同政体(比如奥斯曼君主制和马穆鲁克君主制)的区别由政府改革体现。代码如下:

```
government = monarchy    #君主制
add_government_reform = First_reform #同种政体类型的不同政体
government_rank = 3 #帝国
```

(2) 科技组

这决定了国家初始的科技组, 代码如下:

```
technology_group = eastern #东欧科技组
```

(3) 宗教、文化

决定国家初始的宗教与主流文化, 也可以添加国家的可接受文化, 代码如下:

```
religion = orthodox    #东正教
primary_culture = byrland #拜尔兰自然是拜尔兰人
add_accepted_culture = irish    #接受爱尔兰
```

(4) 首都

此处指定国家首都的所在省份, 用省份编号表示。代码如下:

```
capital = 4118 #省份编号为科克, 在mod中名为科克坦丁堡
```

(5) 国家历史

如果想要这个国家在游戏中的任何时间都存在, 就要在此处编辑君主和继承人的属性。如果想调整之前所有关于国家的内容, 也可以在此处进行修改。代码如下:

```
1453.1.1 = {
    monarch = {
        name = "尼忒弗刻斯"
        dynasty = "森尼尔"
        birth_date = 1425.1.1
        death_date = 1491.1.1
        adm = 2
        dip = 6
    }
}
```

```

        mil = 3
    }
    remove_accepted_culture = irish
}
1491.1.1 = {
    monarch = {
        name = "尼忒芙丝忒一世"
        dynasty = "森尼尔"
        birth_date = 1473.5.29
        female = yes
        adm = 5
        dip = 5
        mil = 5
        female=yes
    }
    add_accepted_culture = irish
}

```

4.2.3 国旗

对应国家的国旗需要创建一张128x128像素的TGA图片文件，并把名字命名为国家tag。这张图片应该放置在：根目录/gfx/flags路径下，如图4-4国旗存放所示：

此电脑 > 文档 > Paradox Interactive > Europa Universalis IV > mod > zishenfox > gfx > flags

名称	修改日期	类型	大小
AZA.tga	2022/3/18 21:05	TGA 图片文件	17 KB
BYR.psd	2022/3/18 21:05	Adobe Photosh...	27 KB
BYR.tga	2022/3/18 21:05	TGA 图片文件	65 KB
EGV.tga	2022/3/18 21:05	TGA 图片文件	49 KB
GLV.tga	2022/3/18 21:05	TGA 图片文件	49 KB
ICA.tga	2022/3/18 21:05	TGA 图片文件	13 KB
MYI.tga	2022/3/18 21:05	TGA 图片文件	7 KB
NGV.tga	2022/3/18 21:05	TGA 图片文件	24 KB

图 4-4 国旗存放

4.3 本地化

EU4当中，本地化的内容文件都被存放在游戏目录下的localization文件夹中，它们的扩展名都是.yml，这表示它们是采用YAML规范编写成的。

除第一行以外，每一行的开头都必须要有额外的空格，标志着这些本地化条目从属于哪种语言。只有双引号里的内容会在游戏当中显示出来。冒号需要和前面的键连在一起，中间不能有空格。

在编辑本地化文件的时候，要注意编码问题。这些文件必须采用带有字节序标记的UTF-8（UTF-8 with BOM）。只有本地化文件使用 UTF-8 with BOM，而游戏文本文件（.txt）以及地图文件夹下的逗号分隔值文件（.csv）应该用 AN

SI 编码（在英文版Windows10当中是指Windows 1252）。如果一个本地化文件用ISO 8859-1（Latin-1）这种单字节字符集编码，那么游戏将会无法读取这个本地化文件，所有的文案在游戏中无法正常显示，就会看到 "Missing Localization!"。

4.4 任务树设计

任务树主要是用于指引玩家的游戏思路，为游玩提供便捷和趣味性，在mod中也可用于推动剧情。任务树的文件位置路径在/mod/mission/中

4.4.1 通用任务树

通用任务树是为官方没有专门编写任务树的国家准备的，一般按照文化、宗教、地域、政体等分类进行划分。任务树的基本结构如下：

```
任务树名称={
    任务树属性
    任务1={
        任务的具体内容
    }
    任务2={
        任务的具体内容
    }
    .....
}
```

（1）任务树的属性部分

任务树的属性部分主要有以下的属性：用于表示此任务树所在的槽位的数值型编码slot，表示是否为通用任务树的逻辑型编码generic，表示ai是否可以完成任务的逻辑型编码ai，表示是否显示国旗盾徽的逻辑型编码has_country_shield，表示是否在该任务树列表中显示的条件代码potential。

（2）单一任务的具体内容

单一任务的具体内容有如下的几项属性：用于显示任务图标(icon)，完成此任务前的前置任务required_missions，显示任务垂直位置的position，显示完成任务所需条件的trigger，提供完成任务奖励的effect。

4.4.2 国家专用任务树

这些是为一些国家向历史方向发展提供便捷的任务树，相对与通用任务树有以下不同：如果希望本任务树覆盖通用的任务树，需要添加代码generic = no; potential一行要写明国家tag（可以写多个）或者首都所在范围；国家专用的任务要添加一行代码has_country_shield = yes；一项任务的历史完成时间写作completed_by =<日期>，这时，如果后期剧本的开局晚于这一天，那么此任务就会被标记为“已完成”，并且得不到奖励；用ai_weight和ai_priority衡量ai在选择任务

树分支时的权重与优先度。

4.4.3 本地化

因为英文源码的标题直接显示并不美观，所以需要任务的名称以及简介进行本地化。由于本地化的需求，所有的中文字体必须进行转码才能显示，为了提高文章可读性，本次展示的是转码之前的代码。如下例所示：

```
l_english:
mission_byr_foothold_title:"立足之地"
mission_byr_foothold_desc:"我们经过漫长的漂流终于来到了爱尔兰群岛，当地的凯尔特人接待了落魄的我们，尽管从千年之前到现在，我们与凯尔特人的关系并不融洽，但重建罗马是头等大事，我们得团结我们所能团结的一切。"
```

4.5 事件设计

事件是用于辅助任务树，以及为游戏增添可玩性的机制，当满足一定的条件时将会自动触发。一个事件主要包含以下几个部分：事件类型、事件id、事件标题与描述、事件图片、触发条件、选项。

4.5.1 事件类型

一个事件可以是关于一个省份的或者是一个国家的，因此有两种事件类型`country_event`和`province_event`。这只是指定了事件根作用域，国家事件也可以具有省份范围效果，反之亦然。

4.5.2 事件 id

事件id必须是唯一的非负整数，不能在任何其他事件文件中复制，无论是自己创建的mod事件，抑或是原版游戏的事件。

为了避免未来官方更新带来的复杂问题，以及确保可靠性和方便维护，使用命名空间是谨慎的做法。在一个事件文件的开头，定义如下所示的命名空间

```
namespace = flavor_byr
```

当定义事件id时，使用命名空间，在它后面加一个点，接着加一个数字，通过命名空间保持了它的独一无二。

```
country_event = {
    id = flavor_byr.1
    ...
}
```

这将使事件id易于管理和避免在调试时需要记住一长串事件id，同时防止P社官方更新以及其它mod里相同事件id带来的冲突，可以跨文件使用相同的命名空

间，只要在每个文件的开头指定这个命名空间。

4.5.3 标题和描述

事件标题和描述文本将在本地化文件里引用，核心代码如下：

```
Title =" flavor_byr.1.t"
Desc = "flavor_byr.1.d"
```

然后在本地化文件里加上如下描述：

```
flavor_byr.1.t:"欢迎使用拜尔兰mod"
```

```
flavor_byr.1.d:"欢迎使用本mod，本mod是B站UP主资深小狐狸的拜占庭系列《跟着君堡去流浪》的衍生mod，推荐各位使用1444年的拜占庭进行游玩"
```

描述文本可以加上限制条件使其在指定条件下显示，但是要确认描述文本是可见的，当游玩tag是byz的国家时，描述将会是 "flavor_byr.1.t.da"。当游玩tag不是byz的国家时，描述将会是"flavor_byr.1.t.db"，核心代码如下：

```
title ="flavor_byr.1.t"
desc = {
    trigger = {
        tag = byz
    }
    desc = "flavor_byr.1.t.da"
}
desc = {
    trigger = {
        NOT = {
            tag = byz
        }
    }
    desc = "flavor_byr.1.t.db"
}
```

4.5.4 事件图片

在picture代码中所选择的图片会出现在事件里，事件图片一般存放在：根目录\gfx\event_pictures里。只要简短地写出文件名字，不需要写出文件路径和后缀名，核心代码如下：

```
picture = BUDDHA_eventPicture
```

和上面所说的描述一样，图片也可以添加条件使其在特定情况下显示。

如果想添加一张自定义的事件图片，需要同时添加指向它的图片（文件夹路径："根目录\gfx\event_pictures"）和 .gfx 文件（文件夹路径："根目录\interface"）

。

4.5.5 触发条件

事件触发的条件是触发器里所有的条件都为真。 `trigger = { }` 里并列的条件默认为“与”操作。可以在触发器作用域的范围之内任意切换。在省份事件里，作用域是一个省份，但是之后就可以转换到拥有这个省份的国家作用域，甚至到其他任意一个国家。

(1) 平均触发时间

平均触发时间的英文是MTTH(`mean time to happen`)，在满足触发条件后，决定事件平均多久触发一次。MTTH里的时间是指平均值，说明这个事件平均触发一次的时间（尽管如此，它还是有可能在下一天就触发，或者一直不触发，这就像自然的概率一样）。这里基础的MTTH可以用天或月来定义。除此之外，还可以用更多的修正语句来进一步变更事件发生的频率，示例如下：如果没有至少0稳定度，那么基础的MTTH就要乘以0.8等于320个月。如果有多个修正所需的条件都为真，那么它们的效果会堆叠乘算。

```
mean_time_to_happen = {  
    months = 400  
    modifier = {  
        factor = 0.8  
        NOT = { stability = 0 }  
    }  
}
```

(2) 事件触发

许多事件没有MTTH，而采用条件：`is_triggered_only = yes`。使用这行代码的时候，事件只能被其它的触发源所触发，通常是另一个事件（有MTTH的事件）或者是一个行动。

4.5.6 选项

在事件中必须给ai或玩家至少一个选项用来关闭事件窗口，当玩家将鼠标放在它上面时，会列出该选项的效果，但设置国家标志除外。每个选项都需要指向文本的本地化。核心代码如下

```
option = {  
    name = history_byr.5.b  
    add_stability = 1  
    add_dip_power = 300  
    add_country_modifier = {  
        name = brief_peace  
        duration = -1  
    }  
}
```

选择该选项将获得1点稳定和300外交点数并且获得一个名为“`brief_peace`”的

修正。

(1) 选项中的触发

在选项也可以添加触发器，判断选项是否可用，必须注意确保始终至少有一个选项可用。这是一个选项中的触发选项的示例，其中只能有一个（但始终是一个）选项可用。在下面的核心代码示例中，选项A仅在威望高于90时才可用，选项B仅在威望低于90时可用。

```
option = {  
    name = "EVTOPTA799"  
    trigger = {  
        prestige = 90 #威望  
    }  
    add_dip_power = 20  
}  
option = {  
    name = "EVTOPTB799"  
    trigger = {  
        NOT = { prestige = 90 }  
    }  
    add_prestige = 20  
}
```

(2) 高亮显示

在事件中的选项中添加代码：`highlight = yes`，将会使其高亮显示。如图4-7选项高亮显示中的第3个选项所示。



图 4-7 选项高亮显示

(3) If 条件

选项可以具有多种效果，这些效果取决于其触发条件是否为真。触发条件在limit大括号中添加，效果在条件满足之后发生。

在下面的核心代码中，如果拥有flag: flash_reback_1 将会设定flag: flash_reback_2并损失两年的税收和人力同时削除flag: flash_reback_1。

如果拥有flag:flash_reback将会设定flag:flash_reback_1并损失一年的税收和人力同时削除flag: flash_reback。

```
if={
  limit = {
    has_country_flag = flash_reback_1
  }
  set_country_flag = flash_reback_2
  add_years_of_income = -2
  add_yearly_manpower = -2
  clr_country_flag = flash_reback_1
}
if = {
  limit = {
    has_country_flag = flash_reback
  }
  set_country_flag = flash_reback_1
  add_years_of_income = -1
  add_yearly_manpower = -1
  clr_country_flag = flash_reback
}
```

(4) ai 意愿

ai意愿决定ai将选择一个选项的机会。ai_chance的数字是因素，如核心代码所示，第一个选项的ai_chance数字为75，第二个选项的ai_chance数字为25，那么ai选择第一个选项的可能性是第二个选项的三倍，

```
option = {
  name = "EVTOPTA1073"          # Increase centralization efforts.
  ai_chance = {
    factor = 75
  }
  add_treasury = -500
}
option = {
  name = "EVTOPTB1073"          # Leave as it is.
  ai_chance = {
    factor = 25
  }
}
```

```

    add_stability = -2
}

```

ai_chance语句也可以添加修正来影响ai意愿，在下面的核心代码中，如果威望为50以上且外交点数为100以上，ai将更倾向选择此选项。

```

ai_chance = {
    factor = 40
    modifier = {
        factor = 0
        NOT = {
            prestige = 50 #威望
        }
    }
}
modifier = {
    factor = 0
    NOT = {
        dip_power = 100 #外交点数
    }
}
}

```

(4) 调试事件

可以在控制台中手动触发事件，即使这个事件条件并没有满足。控制台将输出满足哪些条件（如果有），这在调试时很有用。只需输入event [event id]这行代码就可以触发事件。还可以使用控制台命令testevent [event id]来检查事件触发器是否已满足。

(5) 立即发生的效果

可以让某个效果在事件触发的时候立刻发生，而不用等待玩家的选择。这可以让某个效果的发生与所选择的选项无关。默认情况下，这个效果会被写在事件描述文本下面，也可以加上hidden_effect={}进行隐藏。这将会在事件触发的一天后触发history_byr.2的事件，核心代码如下：

```

immediate = {
    hidden_effect = {
        country_event = {
            id = history_byr.2
            days = 1
        }
    }
}

```

4.6 决议

决议和任务与事件有相似的编写方式，但是比它们拥有更大的逻辑包容性，一般用于制作一些复杂的效果。决议的文件位置在/根目录/mission/中主要有：标

题、显示条件、使用条件、效果、ai意愿等属性。

4.6.1 决议标题

决议的标题具有唯一性，它不能和其他的标题相同。标题可以由使用任意的字母组成，但是使用与决议相关的内容是最简单的。有些决议显示绿色背景而不是蓝色的，需要在标题下加入major=yes的代码，表示其重要性。核心代码如下

```
country_decisions = {
    Gainnewreformmissions = {
        major = yes
        .....
    }
}
```

4.6.2 潜在条件

potential字段决定能够在UI的决议面板看到决议的条件。如果没有满足potential触发条件，则连ai也无法使用该决议，即便allow触发条件得到满足亦然。而且，潜在条件的具体条款，对于玩家是不可见的。如果想要测试潜在条件，可以先把它们写进allow，在游戏中看一下是否符合预期。核心代码如下，这三个拥有的flag是切换到当前类型的任务树设定的，没有要求拥有的flag就是要切换的任务树将要设定的。

```
potential = {
    OR = {
        has_country_flag = byrevent_europe
        has_country_flag = byrevent_colony
        has_country_flag = byrevent_byrlania
    }
    NOT = {
        has_country_flag = byrevent_reform
    }
}
```

4.6.3 允许条件

allow字段决定决议是否能够被玩家或者ai触发，和任务与事件里的trigger类型，假设已经满足所有的potential触发条件。在UI中，高亮的绿色√表示满足条件，当可以触发决议时，游戏中会显示图标提醒。核心代码如下，任务树的切换是随意进行的，所以用always=yes表示总是可以使用。

```
allow = {
    always = yes
}
```

4.6.4 决议效果

effect字段决定触发决议后决议所完成的动作。基本和任务与事件中的一样。核心代码如下，将其余任务树的flag清除，并将要切换的任务树所需的flag设定

上来，最后一行代码就是上文提到用于更新任务树的代码。

```
effect = {  
    set_country_flag = byrevent_reform  
    clr_country_flag = byrevent_europe  
    clr_country_flag = byrevent_colony  
    clr_country_flag = byrevent_byrlania  
    swap_non_generic_missions = yes  
}
```

4.6.5 ai 意愿

此字段给出ai会在何时使用该决议。此字段是可选的。与事件中的ai意愿相似，也一样可以通过设置触发条件来修正ai意愿。核心代码如下

```
ai_will_do = {  
    factor = 1  
    modifier = {  
        factor = 0  
        NOT = { prestige = 50 }  
    }  
}
```

4.6.6 本地化

本地化也同任务与事件类似，在对应的本地化文件里引用，正确的本地化需要转码，这里为了可读性仍使用转码前的文本。如以下代码所示，实际的效果如图4-8任务树变更所示。

```
Gainnewcolonymissions_title:"切换为拜尔兰殖民线任务树"  
Gainnewcolonymissions_desc:"切换为拜尔兰殖民线任务树"  
Gainbyrlaniarmissions_title:"切换为拜尔兰尼亚任务树"  
Gainbyrlaniarmissions_desc:"切换为拜尔兰尼亚任务树"  
Gainnewconquermissions_title:"切换为拜尔兰征服线任务树"  
Gainnewconquermissions_desc:"切换为拜尔兰征服线任务树"  
Gainnewreformmissions_title:"切换为拜尔兰改革线任务树"  
Gainnewreformmissions_desc:"切换为拜尔兰改革线任务树"
```



图 4-8 任务树变更

4.7 灾难

灾难用于模拟国家的衰落和一些特殊事件，它可能是天灾也可能是人祸。一般存放在mod\common\disasters路径下。它有着：名称、显示条件、开始条件、终止条件、灾难进度、结束条件、灾难事件等属性。

4.7.1 灾难名称

灾难名称一般用于在本地化文件中显示，增加游戏的可读性。核心代码如下，后面的注释就是灾难的本地化标题。

```
byr_first_goverment = { #帝国的未来
    .....
}
```

4.7.2 潜在条件

和任务与决议相似，这会决定此灾难是否会出现灾难列表里。核心代码如下，当拥有特殊政府改革“拜尔兰的未来”，且没有终止灾难的flag:had_first_goverment_reform时，灾难将会显示在列表中。

```
potential = {
    has_reform = First_reform
    NOT = {
        has_country_flag = had_first_goverment_reform
    }
}
```

4.7.3 开始条件

这个条件是在满足潜在条件后，什么条件可以开启本灾难。核心代码如下，custom_trigger_tooltip是自定义触发提示，用于在本地化中便捷地显示触发条件，其中的flag: Irish_rebel是游戏过程中会遇上的事件之一，本地化显示为：发生了爱尔兰大叛乱。

```
can_start = {
    has_reform = First_reform
    custom_trigger_tooltip = {
        tooltip = Disaster_first_goverment_Irish_rebel #发生了爱尔兰大叛乱
        has_country_flag = Irish_rebel
    }
}
```

4.7.4 终止条件

定义什么时候本灾难的进程会终止。在本例中，因为该灾难是游戏过程中必然会发生的事情，所以编写代码为always=no永不停止。

```
can_stop = {
    always = no
}
```

4.7.5 灾难的月度增加

这主要是为了给灾难进行预警，以游戏性为主，延缓一些可避免的灾难。在本例中，灾难已经爆发了（上述的大叛乱），所以增长速度很快，以及一些额外的加速选项，如：拥有5笔以上贷款、君主的能力值低于一定程度、政府的腐败

程度高于5。

```
progress = {
  factor = 10
  modifier = {
    factor = 3
    has_reform = First_reform
  }
  modifier = {
    factor = 1
    num_of_loans = 5
  }
  modifier = {
    factor = 1
    NOT = {adm = 4} #君主行政能力
  }
  modifier = {
    factor = 1
    NOT = {dip = 4} #君主外交能力
  }
  modifier = {
    factor = 1
    NOT = {mil = 4} #君主军事能力
  }
  modifier = {
    factor = 2
    corruption = 5
  }
}
```

4.7.6 结束条件

这是让灾难结束的条件。只有满足它才能结束这场灾难，核心代码如下：拥有1点稳定；没有叛军和叛军占据的省份；该灾难已经持续了五年；已经决定出新的政府改革。

```
can_end = {
  stability = 1
  num_of_rebel_armies = 0
  num_of_rebel_controlled_provinces = 0
  OR={
    custom_trigger_tooltip = {
      tooltip = Disaster_first_goverment_had_first_goverment_reform
      #该灾难至少持续了五年
    }
    had_country_flag = {
      flag = Feudalism_vs_Autocracy_goverment_reform
      days = 1825
    }
  }
}
```

```

    }
  }
  has_reform = Rule_of_Senate_reform
  has_reform = Upper_House_election_reform
  has_reform = Emperor_centralization_reform
}
}

```

4.7.7 灾难事件

灾难事件由两个事件组成，一个是灾难开始的事件，一般会提供大量负面效果。一个是灾难结束的事件，一般会提供正面的效果，并且让该灾难不会再次发生。核心代码如下。

```

on_start = first_government_reform.1
on_end = first_government_reform.2

```

4.7.8 月度事件

在灾难期间，每个月都会判定触发的事件，可以通过事件的权重来调整触发的频率，如核心代码所示，每月没有强制触发的事件，有四分之三的概率推进政府的改革进度，十六分之三的概率无事发生，十六分之一的概率发生坏事。

```

on_monthly = {
  events = {
  }
  random_events = {
    350 = 0
    50 = losing_control.1
    200 = first_government_reform.3
    200 = first_government_reform.4
    200 = first_government_reform.5
    200 = first_government_reform.6
  }
}

```

4.7.9 本地化

对灾难的名称和描述进行本地化。如以下核心代码所示。正确的本地化需要转码，这里为了可读性仍使用转码前的文本。

```

l_english:
  disaster_BYR_religion_reform_war:"拜尔兰宗教改革战争"
  desc_disaster_BYR_religion_reform_war:"拜尔兰宗教改革战争"
  byr_first_government:"帝国的未来"
  desc_byr_first_government:"帝国的未来"
  Disaster_first_government_Irish_rebel:"发生了爱尔兰大叛乱"
  disaster_active_for_15_years_tooltip:"灾难至少持续了十五年"

```

4.7.10 灾难图标

灾难图标的位置在mod/gfx/interface/disasters文件夹中。一个图标文件内要做三个版本的图标：进度增长时的彩色标志、爆发后，带有火光的标志，以及潜在灾难的灰色标志。最后在mod/interface/countrystabilityview.gfx添加如下核心代码，让图标出现在稳定与扩张界面中。

```
spriteType = {  
    name = "GFX_disaster_disaster_BYR_religion_reform_war"  
    texturefile = "gfx//interface//disasters//religious_turmoil.dds"  
    noOfFrames = 3  
    loadType = "INGAME"  
}
```

4.8 项目代码

4.8.1 任务树代码

前期任务树的核心代码如下，这个任务树要求必须使用国家：拜占庭进行游戏，为了防止和原版游戏的任务树冲突，设定在游戏开局一段时间之后触发事件给予flag:breland_start_flag用于开启任务树。

```
byz_retreat={          #任务树代号  
    potential= {        #显示任务的条件  
        tag = BYZ #国家=拜占庭  
        has_country_flag = breland_start_flag  
        #已拥有flag:breland_start_flag  
    }  
    start_retreat = {          #任务名  
        icon = mission_rb_a_mighty_fleet #图像  
        required_missions = {}          #之前需要完成的任务  
        trigger = {                #完成任务所需要的条件  
            army_size_percentage = 1  
            num_of_generals = 1  
        }  
        effect = {                #完成任务的效果  
            add_treasury = 300  
            add_country_modifier = {  
                name = "start_retreat"  
                duration = 3650  
            }  
            4118={ add_claim = BYZ }  
        }  
    }  
}
```

4.8.2 事件代码

为了方便任务树的完成和完成后的运行，笔者对之后发生的事进行了模拟，制作了一些“历史”事件，增加可玩性，以下面的事件代码为例，第一个是省份

事件，拜占庭和拜尔兰同时存在时，在爱尔兰岛上发生了战争，拜占庭或拜尔兰的军队攻陷了一个省份，这个省份就会给它的控制者发送第二份事件。让这个省份的拥有者直接割让省份给拜尔兰，并获得一定数量的杜卡特金币和威望。

```
province_event = {
    .....
    trigger = {
        OR={          #爱尔兰岛上的四个区域
            area = ulster_area
            area = munster_area
            area = connacht_area
            area = leinster_area
        }
        NOT = { controlled_by = owner } #当控制者不是拥有者
        any_neighbor_province = {      #作用域：任意周围省份
            owned_by = BYR  #拜尔兰
            owned_by = BYZ  #拜占庭
        }
    }
    option = {
        name = flavor_byz.3.a
        owner = { add_prestige = -1 }
        controller = { country_event = { id = flavor_byz.4 } }
    }
}
country_event = {
    .....
    option = {
        name = flavor_byz.4.a
        FROM = { #事件的接收方，
            cede_province = BYR
        }
        add_treasury = 50
        add_prestige = 1
    }
}
```

4.8.3 决议代码

这个决议是用于给情怀向玩家使用的，当1460年以后，主流文化是拜尔兰人的君主制独立国家，当其王室不是森尼尔家族时，可以尝试迎回其主脉，更换当前君主与继承人。核心代码如下。

```
enthroned_byreland_prince = {
    potential = {
        is_year = 1460
```

```

    NOT = { dynasty = "森尼尔" }
    government = monarchy
    OR = {
        culture_group = Byzantine
        primary_culture = byrland
    }
    is_lesser_in_union = no
    Is_vassal = no
}
allow = {
    NOT = { dynasty = "森尼尔" }
}
effect = {
    define_ruler = {
        dynasty = "森尼尔"
        culture = byrland
    }
    if = {
        limit = {
            has_heir = yes
        }
        remove_heir = yes
    }
    define_heir = {
        dynasty = "森尼尔"
        culture = byrland
    }
}
}
}

```

4.8.4 灾难代码

这些核心代码主要是用于为新设定的宗教所处理的，当已经转为新教之后，就会开始灾难进度的读取。这个灾难是必然触发的，要结束这个灾难需要：没有叛军和叛军占领的省份、宗教统一达到80%、稳定度达到1、灾难持续了10年。每个月都会有概率触发坏事件，增大挑战性，另一个事件是通过事件的连锁触发形成的事件组，只会触发一次。

```

disaster_BYR_religion_reform_war = {      #拜尔兰宗教改革战争
.....
can_end = {
    num_of_rebel_armies = 0
    num_of_rebel_controlled_provinces = 0
    num_of_revolt = 0
    stability = 1
    religious_unity = 0.8
}
}

```

```
        had_country_flag = {
            flag = has_WAR_religion_reform
            days = 3650
        }
    }
    .....
    on_monthly = {
        events = {
        }
        random_events = {
            1000 = 0
            100 = losing_control.1
            100 = byrreligion_reform.101
        }
    }
}
```

5 mod测试

5.1 功能测试

mod功能测试是为了验证本mod是否能够实现基本功能，即从君士坦丁堡转进到爱尔兰，本mod以任务树驱动过程，完成任务树即可完成转进。

(1) 开始游戏

事件是锁定国家tag的，不用担心会不触发，选择第一项将会进入转进路线，选择第二项则会进入挑战路线。顺序如图5-1(1)旧任务树、图5-1(2)更换任务树事件、图5-1(3)新任务树所示：



图5-1(1) 旧任务树

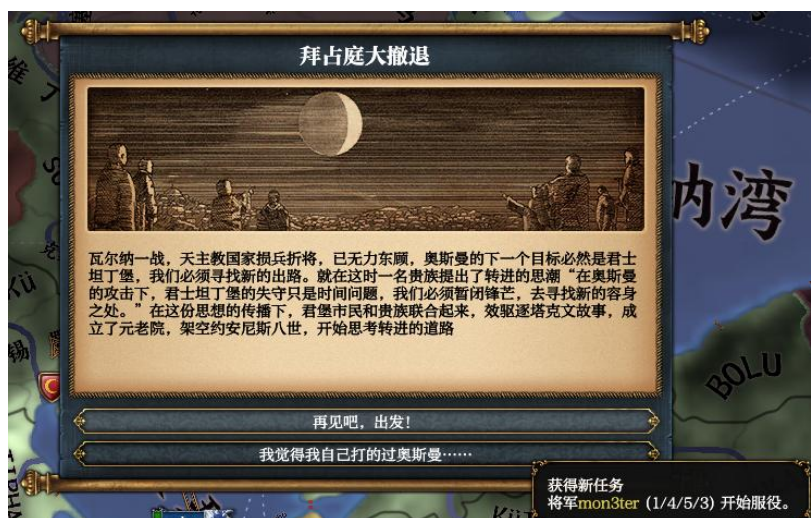


图5-1(2) 更换任务树事件



图5-1(3) 新任务树

(2) 逐步完成任务树

这会是一个很轻松地游玩过程，右二的任务树是通用任务树，完成与否和转进任务树无关。完成最后的任务，将会提供一个事件用以转移游玩的国家。如图5-2(1)完成任务树、5-2(2)转进事件所示：



图5-2(1) 完成任务树



图5-2(2) 转进事件

(3) 完成任务树之后

点击选项使转进事件结束，玩家扮演的国家变更为拜尔兰的同时，获得切换任务树的决议和新国家任务树，如图5-3(1)切换任务树决议、图5-3(2)新国家任务树所示：



图5-3(1) 切换任务树决议



图5-3(2)新国家任务树

5.2 稳定性测试

本mod在自行的稳定性测试过程中，多次出现乱码、图片丢失等情况，一般是在本地化编码和界面设置中出现了错误，经过修改后系统正常运行，未出现报错。在上传至steam创意工坊后根据玩家的反应查漏补缺，逐步完善mod。因为在此期间官方进行了版本更新，所以要根据新版本的变动重新编写部分代码。在不发生版本变动的情况下，稳定性很强，在发生版本变动时，稳定性较弱。需要不断关注并维护更新。
